SRS — Micro Donation Platform

1. Introduction

1. Introduction

1.1 Purpose

This SRS defines the functional and non-functional requirements for a Micro Donation Platform — a web application that lets users donate small amounts (₹1–₹500) to verified causes and individuals.

1.2 Scope

The platform covers donor registration, cause discovery, payment processing, campaign management, and impact reporting. Out of scope: physical fundraising, non-digital payments, and regulatory compliance beyond India (UPI/Razorpay).

1.3 Document Conventions

- **FR-xx**: Functional Requirement
- **NFR-xx**: Non-Functional Requirement
- **UC-xx**: Use Case
- Priority levels: P1 (must-have), P2 (should-have), P3 (nice-to-have)

1.4 Intended Audience

Developers, testers, and the SE course evaluator. Assumes basic familiarity with web applications.

2. Overall Description

2. Overall Description

2.1 Product Perspective

A standalone web platform (React + Node.js) integrated with UPI/Razorpay for payments and a PostgreSQL backend. Runs on the public internet; no hardware dependencies.

2.2 Product Features (Summary)

- Donor account creation and authentication
- Campaign creation by verified cause owners
- Micro-donation processing (₹1–₹500 per transaction)
- Real-time donation tracker and impact dashboard
- Admin panel for campaign verification and moderation

2.3 User Classes

User	Description	Access
Guest	Unauthenticated visitor	Browse campaigns only
Donor	Registered user who donates	Donate, track history
Campaign Owner	NGO/individual running a cause	Create & manage campaigns
Admin	Platform moderator	Verify, approve, suspend accounts/campaigns

2.4 Operating Environment

- Browser: Chrome, Firefox, Safari (latest 2 versions)
- Backend: Node.js 20 LTS on a cloud VM (AWS/GCP)
- Database: PostgreSQL 15
- Payment: Razorpay Test/Live API
- CI/CD: GitHub Actions

3. Functional Requirements

3. Functional Requirements

3.1 User Authentication

ID	Requirement	Priority
FR-01	Users shall register with email, password, and display name	P1
FR-02	Users shall log in via email/password or Google OAuth	P1
FR-03	System shall send email verification on registration	P1
FR-04	Users shall be able to reset password via email link	P1
FR-05	Sessions shall expire after 7 days of inactivity	P2

3.2 Campaign Management

ID	Requirement	Priority
FR-06	Campaign owners shall create campaigns with title, description, goal amount, and deadline	P1
FR-07	Campaign owners shall upload supporting images/documents	P2
FR-08	Admin shall approve or reject submitted campaigns within the platform	P1
FR-09	Campaign owners shall be able to post progress updates	P2
FR-10	System shall auto-close campaigns past their deadline	P1

3.3 Donation Processing

ID	Requirement	Priority
FR-11	Donors shall select a campaign and enter an amount between ₹1 and ₹500	P1
FR-12	System shall process payment via Razorpay (UPI, card, net banking)	P1
FR-13	System shall generate and email a donation receipt on success	P1
FR-14	Donors shall be able to make anonymous donations	P2
FR-15	Donations shall be refundable within 24 hours if the campaign is rejected post-donation	P2

3.4 Discovery & Browse

ID	Requirement	Priority
FR-16	Guests and donors shall browse campaigns by category, location, and urgency	P1
FR-17	System shall display a real-time donation progress bar per campaign	P1
FR-18	System shall recommend campaigns based on a donor's past donation categories	P3

3.5 Dashboard & Reporting

ID	Requirement	Priority
FR-19	Donors shall view their donation history with amounts, dates, and campaign names	P1
FR-20	Campaign owners shall view total funds raised, donor count, and timeline chart	P1

ID	Requirement	Priority
FR-21	Admin shall view platform-wide metrics: total donations, active campaigns, flagged items	P1
FR-22	System shall export donor history as a PDF or CSV	P2

4. Non-Functional Requirements

4. Non-Functional Requirements

ID	Category	Requirement	Measure
NFR-01	Performance	Page load time under normal load	< 2 seconds (95th percentile)
NFR-02	Performance	Payment processing response	< 5 seconds end-to-end
NFR-03	Scalability	Concurrent users supported without degradation	⩾ 500
NFR-04	Availability	Platform uptime	⩾ 99.5% monthly
NFR-05	Security	All data in transit encrypted	TLS 1.2+
NFR-06	Security	Passwords stored using bcrypt	Work factor ⩾ 12
NFR-07	Security	Payment data never stored locally	PCI-DSS via Razorpay tokenization
NFR-08	Usability	Core donation flow completable without instructions	⩽ 3 clicks from campaign page
NFR-09	Usability	UI responsive on mobile and desktop	Viewport ⩾ 360px
NFR-10	Maintainability	Test coverage for backend API	⩾ 70% line coverage

ID	Category	Requirement	Measure
NFR-11	Portability	Deployable via Docker on any Linux host	Single `docker-compose up`

5. System Architecture Overview

5. System Architecture Overview

3-tier architecture: React SPA → REST API (Node.js/Express) → PostgreSQL. Razorpay handles all payment flows externally.

Layer	Technology	Responsibility
Frontend	React 18, Tailwind CSS	UI, routing, state management
Backend API	Node.js 20, Express	Business logic, auth (JWT), REST endpoints
Database	PostgreSQL 15	Users, campaigns, donations, transactions
Payment	Razorpay API	Payment initiation, webhook callbacks
Email	Nodemailer + SMTP	Receipts, verification, notifications
Hosting	Docker + GitHub Actions	CI/CD, containerised deployment

Key API modules: `/auth`, `/campaigns`, `/donations`, `/admin`, `/users`

All frontend-to-backend calls use JWT bearer tokens. Razorpay webhooks are verified via HMAC signature before updating donation status.

6. Use Cases

6. Use Cases

UC-01: Donor Makes a Micro Donation

Actor: Donor **Precondition:** Donor is logged in; campaign is active.

1. Donor browses campaigns and selects one.

2. Donor clicks Donate, enters amount (₹1–₹500).
3. System opens Razorpay checkout.
4. Donor completes payment.
5. System confirms payment via webhook, updates progress bar, emails receipt.

Alternate: Payment fails → system shows error, donation not recorded.

UC-02: Campaign Owner Creates a Campaign

Actor: Campaign Owner **Precondition:** Owner account is verified.

1. Owner navigates to Create Campaign.
2. Fills in title, story, goal, category, deadline.
3. Uploads supporting documents.
4. Submits — status set to **Pending Review**.
5. Admin reviews and approves/rejects.
6. On approval, campaign goes live.

UC-03: Admin Moderates a Campaign

Actor: Admin **Precondition:** At least one campaign in Pending Review.

1. Admin opens the moderation queue.
2. Reviews details and documents.
3. Approves (campaign goes live) or Rejects (owner notified with reason).

UC-04: Donor Views Impact Dashboard

Actor: Donor **Precondition:** Donor has at least one completed donation.

1. Donor navigates to My Impact.
2. System shows total donated, number of campaigns supported, and categories breakdown.

7. Constraints & Assumptions

7. Constraints & Assumptions

Technical Constraints

- Payment gateway limited to Razorpay (India); international cards are a stretch goal.
- Max donation per transaction: ₹500 (platform policy, not a technical limit).
- File uploads (campaign images) capped at 5 MB per file.

Business Constraints

- Platform takes 0% cut in the MVP; payment gateway fees (~2%) apply.
- Campaigns must be manually approved by admin before going live.

Assumptions

- All users have internet access and a UPI-linked device or card.
- Team uses Razorpay test mode throughout development; live keys added before demo.
- The platform is a course project — no real money will be processed.

8. Glossary

8. Glossary

Term	Definition
Micro Donation	A monetary contribution of ₹1–₹500 per transaction
Campaign	A fundraising page created by a verified cause owner
Campaign Owner	An NGO or individual who creates and manages a campaign
Donor	A registered user who contributes money to campaigns
Admin	A platform moderator with privileges to approve/reject campaigns and suspend accounts

Term	Definition
Razorpay	Third-party Indian payment gateway handling UPI, card, and net banking transactions
Webhook	HTTP callback from Razorpay to the backend confirming payment status
JWT	JSON Web Token used for stateless user authentication
FR	Functional Requirement
NFR	Non-Functional Requirement